

Data Transfer and Addressing

ELF memory maps, load/store architecture, addressing modes, and string/array traversal in AArch64

Computer Organization & Architecture | AArch64 Reference

Topic 6 Outline

Topic Objective

Analysis of program memory organization (ELF layouts), AArch64 load-store architecture, memory addressing modes, alignment constraints, and pointer-based data traversal.

- **Section 1:** Program Memory Organization (ELF Layouts)
- **Section 2:** The Load-Store Architecture Paradigm
- **Section 3:** Load and Store Instructions (`ldr`, `str`)
- **Section 4:** AArch64 Memory Addressing Modes
- **Section 5:** Memory Access, Alignment, and Pair Transfers
- **Section 6:** Traversing Arrays, Pointers, and Strings
- **Section 7:** Summary and Worked Self-Test Exercises

Program Memory Organization

The ELF Memory Layout Paradigm

What is ELF?

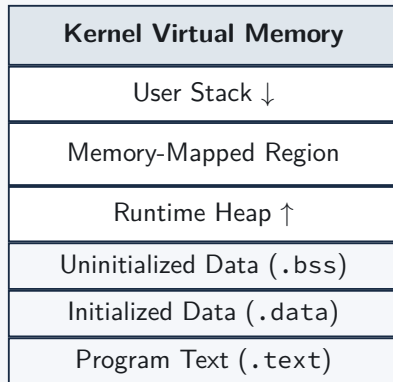
ELF (Executable and Linkable Format) is the standard binary file format for executables, object code, and shared libraries in Unix-like systems.

- Defines how code, static data, and linking symbols are organized on disk and loaded into memory.
- Contains processor-specific flags, such as machine type identification and alignment requirements.
- When an ELF binary executes, the operating system maps it into virtual memory segments.

The Linux Virtual Memory Map

- **Kernel Space:** Reserved space for kernel code, process tables, and kernel stacks.
- **User Stack:** Grows downward; stores local variables and activation frames.
- **Shared Libraries:** Mapped memory regions for dynamic libraries.
- **Runtime Heap:** Grows upward via dynamic allocation (`malloc`).
- **BSS / Data / Text:** Uninitialized, initialized, and executable code segments.

Process Virtual Memory



Load-Store Architecture

Load-Store Architecture Principles

AArch64 follows a strict RISC load-store design philosophy:

- **Separation of Concerns:** Computation instructions (add, sub) process register operands exclusively. Memory instructions (ldr, str) handle data movement.
- **No In-Memory Arithmetic:** An instruction cannot simultaneously fetch from memory, add, and write back.
- **Simplified Datapath:** Keeps hardware control logic streamlined, highly predictable, and optimized for deep pipelining.

C Code Implication

C statements like `a = b + c;` require explicit loading of variables from memory into registers, addition, and a final store instruction.

Memory Addresses as Register Values

Memory locations are targeted using 64-bit virtual addresses held in general-purpose registers:

- The base address of a variable, array, or data structure is loaded into an x register (e.g., x1).
- Load/store instructions treat this register as a pointer referencing the target byte location.
- Pointer arithmetic can manipulate base addresses dynamically.

Pointer Register Rule

Only 64-bit general-purpose registers (x0–x30) can serve as base address pointers in AArch64.

Load and Store Instructions

Loading Data: The `ldr` Instruction

The Load Register (`ldr`) instruction copies data from memory into a target register:

- **Syntax Format:** `ldr destination_reg, [base_register]`
- **64-bit Transfer:** `ldr x0, [x1]` loads 8 bytes from the address in `x1` into `x0`.
- **32-bit Transfer:** `ldr w0, [x1]` loads 4 bytes into `w0`, zero-extending the upper 32 bits of `x0`.

Data Sizing Suffixes

Sub-word loads are supported via explicit qualifiers like `ldrb` (byte) and `ldrh` (halfword), with sign-extended variants (`ldrshb`, `ldrshh`).

Storing Data: The `str` Instruction

The Store Register (`str`) instruction copies data from a register into memory:

- **Syntax Format:** `str source_reg, [base_register]`
- **64-bit Transfer:** `str x0, [x1]` writes the 8-byte contents of `x0` to the memory address held in `x1`.
- **32-bit Transfer:** `str w0, [x1]` writes the lower 4 bytes of `w0` to memory.

Sub-Word Stores

Corresponding store variants `strb` (store byte) and `strh` (store halfword) allow granular writing of lower register bytes to memory destinations.

Precision Data Sizing and Sign Extension

AArch64 load instructions allow precise control over data width and extension behavior:

Instruction	Bytes Loaded	Extension Behavior
<code>ldrb w0, [x1]</code>	1 Byte (8 bits)	Zero-extended to 32 bits in w0
<code>ldrsb x0, [x1]</code>	1 Byte (8 bits)	Sign-extended to 64 bits in x0
<code>ldrh w0, [x1]</code>	2 Bytes (16 bits)	Zero-extended to 32 bits in w0
<code>ldrsh x0, [x1]</code>	2 Bytes (16 bits)	Sign-extended to 64 bits in x0

Sign Extension Importance

When loading signed 8-bit or 16-bit integers into larger registers, using sign-extended variants (`ldrsb/ldrsh`) preserves negative values correctly.

Memory Addressing Modes

Addressing Mode Classification

An addressing mode defines how the CPU calculates the target memory address for a load or store instruction:

- **Base Register (Indirect):** Address is taken directly from the base register.
- **Immediate Offset:** Adds a constant offset to the base register without modifying the base.
- **Register Offset:** Adds a scaling or offset register to the base address.
- **Pre-Indexed:** Applies an offset, updates the base register, and accesses memory.
- **Post-Indexed:** Accesses memory using the base address, then updates the base register.

Base Register (Indirect) Addressing

Syntax and Semantics

```
ldr x0, [x1]
```

The memory address is taken strictly from the contents of the base register x1. The base register remains unchanged after execution.

- **Use Case:** Accessing a variable or dereferencing a pointer whose exact address is already loaded in a register.
- Analogous to pointer dereferencing in C: `*ptr`.

Immediate Offset Addressing

Syntax and Semantics

```
ldr x0, [x1, #16]
```

Computes the target address by adding an immediate constant offset (+#16) to the base register x1. The base register x1 is **not** modified.

- **Use Case:** Accessing fields within a structure or fixed offsets in a local stack frame.
- Immediate range rules depend on transfer size.

Pre-Indexed Addressing Mode

Syntax and Semantics

```
ldr x0, [x1, #8]!
```

The immediate offset (+#8) is added to base register x1. This new calculated address is used for the memory load, **and** is written back into x1.

- Identified by the trailing exclamation mark (!).
- **Use Case:** Updating a pointer before accessing the next sequential memory item.

Post-Indexed Addressing Mode

Syntax and Semantics

```
ldr x0, [x1], #8
```

Data is loaded from the unmodified address in x1. After the memory access completes, the offset (+#8) is added to x1, updating the pointer register.

- Identified by placing the offset outside the square brackets.
- **Use Case:** Fetching current array elements and automatically advancing the pointer for the next iteration.

Register Offset and Scaled Addressing

AArch64 allows an index register to offset the base pointer, optionally scaled by data size:

- **Syntax:** `ldr x0, [x1, x2]` → Address is $x1 + x2$.
- **Scaled Register Offset:** `ldr x0, [x1, x2, lsl #3]` → Address is $x1 + (x2 \times 8)$.

Array Indexing Power

Scaling shifts the index register by the exact byte width of the data item ($2^3 = 8$ bytes for doublewords), mapping C array subscripts directly into hardware addressing.

Addressing Mode Summary

Mode	AArch64 Assembly	Address	Base Register
Offset	<code>ldr x0, [x1, x2]</code>	$x1 + x2$	Unchanged
Pre-index	<code>ldr x0, [x1, x2]!</code>	$x1 + x2$	$x1 = x1 + x2$
Post-index	<code>ldr x0, [x1], x2</code>	$x1$	$x1 = x1 + x2$

Note on Register vs. Immediate

Offset and index quantities can be specified as general-purpose registers, as shown above, or compatible immediate byte constants such as #8.

Memory Access, Alignment, and Pair Transfers

Memory Alignment Rules in AArch64

Data alignment dictates how memory addresses map to data sizes:

- **Natural Alignment:** An n -byte data item should be stored at an address divisible by n .
 - Doublewords (8 bytes) are naturally aligned at addresses divisible by 8.
 - Words (4 bytes) are naturally aligned at addresses divisible by 4.
 - Halfwords (2 bytes) are naturally aligned at addresses divisible by 2.
- **Performance and Correctness:** Alignment requirements depend on the particular access and system configuration; naturally aligned accesses are generally preferred.

Load and Store Pair Instructions (**ldp** / **stp**)

AArch64 provides specialized instructions to transfer **two registers** with one instruction:

- **Load Pair (**ldp**)**: `ldp x0, x1, [x2]` loads two 64-bit registers from consecutive memory locations starting at `x2`.
- **Store Pair (**stp**)**: `stp x0, x1, [x2]` stores two 64-bit registers consecutively to memory.

Instruction-Count Benefit

Pair instructions move two register values with one architectural instruction and are especially useful for stack-frame save/restore operations.

Stack Operations via Pair Transfers

Stack management relies heavily on pre-index and post-index pair instructions:

Pushing to the Stack (Pre-Index)

```
stp x29, x30, [sp, #-16]!
```

Decrements stack pointer (sp) by 16 bytes, then stores frame pointer (x29) and link register (x30).

Popping from the Stack (Post-Index)

```
ldp x29, x30, [sp], #16
```

Loads registers from current sp, then increments stack pointer by 16 bytes, restoring stack balance.

Section 06

Traversing Arrays, Pointers, and Strings

Initializing Base Pointers in C and Assembly

C Pointer Initialization

- `int array[6] = {2, 3, 11, 4, 55, 6};`
- `int *ptr = array; // base pointer`
- `*(ptr += 4) += 10;`

AArch64 Base Register Setup

- Declare static data arrays in the `.data` section.
- Load the array base address into a pointer register using symbol directives:
- `ldr x0, =array`

Mapping C Arrays to AArch64 Assembly

Arrays consist of contiguous memory locations holding elements of identical size:

- Let `x1` hold the base address of an integer array (`int A[]`, where each element is 4 bytes).
- Accessing element `A[i]` requires computing the effective address:

$$\text{Base} + (\text{Index} \times \text{Element Size})$$

- Assembly implementations can use scaled register offsets or explicit pointer arithmetic.

Example Array Access

`ldr w0, [x1, x2, lsl #2]` loads `A[i]` into `w0`, where `x2` holds index `i` and the shift multiplies it by 4.

Iterating Over Strings (.asciz)

Strings in C/C++ are represented as contiguous character arrays terminated by a null byte (0):

- Declared using assembler directives such as `message: .asciz "Hello World!"`.
- Accessing individual characters requires byte-level instructions such as `ldrb`.
- Using a post-indexed byte load, `ldrb w1, [x0], #1`, fetches the current character and then advances the pointer.

Loop Termination Check

The loop continues reading until the loaded character equals zero, for example with `cmp w1, #0` followed by `b.eq end`.

Summary & Self-Test

Topic 6 Comprehensive Summary

- **ELF Memory Layout:** Programs execute across mapped virtual regions including text, data, BSS, heap, mapped regions, and stack.
- **Load-Store Paradigm:** Memory access uses explicit load/store instructions; arithmetic and logical computation operates on registers.
- **Addressing Modes:** AArch64 supports base, immediate offset, register offset, pre-indexed, and post-indexed addressing forms.
- **Alignment & Pairs:** Natural alignment is preferred, while `ldp` and `stp` provide efficient pair transfers.
- **Data Structures:** Scaled offsets, pointers, and byte loads support array and string traversal.

Self-Test: Questions 1 & 2

Question 1: ELF Segments

What is the primary difference between the `.text` segment and the `.data` segment in an ELF executable?

Question 2: Immediate vs. Indexed Addressing

What is the primary operational difference between immediate offset addressing (`[x1, #8]`) and pre-indexed addressing (`[x1, #8]!`)?

Self-Test: Solutions 1 & 2

Solution 1

The `.text` segment stores the program's executable machine instructions, while the `.data` segment stores initialized global and static variables.

Solution 2

Immediate offset adds the constant to the base register solely to calculate the target address, leaving the base register unchanged. Pre-indexed addressing calculates the address **and writes the updated address back** into the base register.

Self-Test: Questions 3 & 4

Question 3: Post-Indexed String Traversal

How does the instruction `ldrb w1, [x0], #1` simplify iterating through a null-terminated string compared with using a separate addition instruction?

Question 4: Memory Alignment

Why is natural alignment generally preferred for memory accesses?

Self-Test: Solutions 3 & 4

Solution 3

It loads the byte at the current address in x0 into w1 and then increments x0 by 1 as part of the same architectural instruction, eliminating a separate `add x0, x0, #1`.

Solution 4

Natural alignment matches the data size to an appropriately aligned address and generally avoids unnecessary access complexity or implementation-dependent penalties.

Self-Test: Questions 5 & 6

Question 5: Pair Transfers on the Stack

How do `stp` and `ldp` manage stack pointer updates during function entry and exit?

Question 6: Scaled Register Addressing

In the instruction `ldr x0, [x1, x2, lsl #3]`, what role does the `lsl #3` modifier play in indexing an array of 64-bit integers?

Self-Test: Solutions 5 & 6

Solution 5

Using pre-indexed store pair (`stp ... [sp, #-16]!`), the stack pointer is decremented before the registers are stored. Post-indexed load pair (`ldp ... [sp], #16`) restores the registers and then increments the stack pointer.

Solution 6

The `lsl #3` scales the index in `x2` by multiplying it by $2^3 = 8$, the byte size of a 64-bit element, translating an array index into a byte offset.

Review: Addressing Mode Summary Table

Addressing Mode	Assembly Syntax Example	Base Register State After Access
Base Register	<code>ldr x0, [x1]</code>	Unchanged
Immediate Offset	<code>ldr x0, [x1, #16]</code>	Unchanged
Pre-Indexed	<code>ldr x0, [x1, #8]!</code>	Updated ($\text{Base} + 8$)
Post-Indexed	<code>ldr x0, [x1], #8</code>	Updated ($\text{Base} + 8$)

Wrap-Up and Next Steps

Transition to Topic 7

Building upon data transfer and memory addressing mechanics, the next module examines:

- **Branching and Iteration:** Managing program flow and dynamic execution paths.
- **Comparison Instructions:** Evaluating operand relationships to set processor condition flags.
- **Conditional and Unconditional Branches:** Controlling execution jumps based on flag states or targets.
- **Loops and Selection Structures:** Implementing constructs such as `if-else` branches and iterative loops.
- **Translation of High-Level Control Flow:** Converting high-level programming logic into assembly routines.